Redis

Redis = **Remote Dictionary Server**

Redis is an in-memory key-value data store used for caching, messaging, rate limiting, and real-time applications because it provides extremely low latency and high throughput.

Normal database:

App → Database (disk) → slower

With Redis:

App → Redis (RAM) → extremely fast
App → Database (disk) → slower but persistent

Key properties

- **In-memory** → data stored in **RAM**
- **Key–value store**
- **Extremely fast (microseconds latency)**
- Supports **data structures** (not just strings)

Data structures Redis supports

- String
- Hash
- List
- Set
- Sorted Set
- Bitmap
- HyperLogLog
- Stream

Why Redis is used

- Very **low latency**
- Handles **millions of ops/sec**
- Perfect for **temporary / frequently accessed data**

1. Why Redis is so fast ⚡

In-memory storage

Most databases read from **disk (SSD/HDD)**.

Disk access ≈ **milliseconds**
RAM access ≈ **nanoseconds**

So Redis avoids disk I/O.

App → Redis (RAM) → very fast
App → DB (Disk) → slower

Simple data model (Key-Value)

Example:

SET user:1 "Prajwal"
GET user:1

Lookup is basically a **hash table operation → O(1)**.

Very little computation needed.

Single-threaded command execution

Redis processes commands **one at a time**.

So it **avoids locks, thread contention, and context switching**.

Request 1
Request 2
Request 3

Request 4

↓

Event Loop executes sequentially

This makes Redis extremely predictable and fast.

Event-driven architecture (epoll / kqueue)

Redis uses **non-blocking I/O**.

Meaning it can manage **thousands of connections** without creating threads.

Client connections

↓

Event loop

↓

Execute commands

Similar concept used in **Node.js**.

How Redis stores data internally

At the core Redis stores everything like:

key → value

Example:

"user:1" → "Prajwal"

Internally Redis uses a **Hash Table (Dictionary)**.

Hash Table

index → key → value

Operations:

Operation	Complexity
GET	O(1)
SET	O(1)
DEL	O(1)

That's why Redis is extremely fast.

3. Redis Persistence (RDB vs AOF)

Since Redis stores data in **RAM**, data would be lost on restart.

So Redis provides **persistence options**.

RDB (Redis Database Snapshot)

Redis saves **snapshots of memory to disk** periodically.

Example:

Redis memory
↓
Snapshot
↓
dump.rdb file

Example config:

save 900 1

Meaning:

If **1 key changes in 900 seconds**, create snapshot.

Pros

- Smaller file
- Faster restart
- Good for backups

Cons

- Data loss possible between snapshots.

AOF (Append Only File)

Redis logs **every write operation**.

Example:

SET user:1 Prajwal
INCR counter
LPUSH queue job1

Saved to:

appendonly.aof

On restart Redis **replays the commands**.

Pros

- Very little data loss
- Better durability

Cons

- Larger file
- Slightly slower

Production setup usually

Redis
├─ RDB (backup)
└─ AOF (durability)

4. Redis Single Thread Architecture

Redis mainly uses **one thread for command execution**.

Flow:

Clients
↓
Socket connections
↓
Event Loop
↓
Command execution
↓
Response

Why single thread works

Because operations are:

- very fast
- memory based
- Yesnon-blocking

So Redis can still handle **100k+ requests/sec**.

Modern Redis versions also use **threads for networking I/O**, but command execution is still mostly single-threaded.

Caching

The Idea of Caching

If the data **does not change frequently**, we can store it in **Redis**.

Instead of:

Client → DB

We do:

Client → Cache → DB

Flow:

1. Check Redis
2. If exists → return
3. If not → query DB
4. Save in Redis
5. Return response

This is called:

Cache Aside Pattern (Lazy Loading)

This is the **most common caching strategy**.

Steps to connect to Redis

1. Install redis client:

npm install redis

Make sure Redis server is running.

2. Run Redis using Docker (very common in real projects)

Many backend devs prefer this because **no installation needed**.

Run:

docker run -d -p 6379:6379 redis

What this does:

Docker Container

Redis Server

localhost:6379

Your Node app still connects the same way.

OR

Redis Cloud (production)

In production:

AWS Elasticache
Upstash
Redis Cloud

Then your server connects via URL.

3. connect redis

```
import express, {Request,Response} from "express";
import {createClient} from "redis";

const client= createClient({
   url:"redis://localhost:6379"
});

client.on("error", (err)=> console.log("Redis client error", err));
```

```
async function connectRedis(){
   await client.connect();
   console.log("Connected to Redis");
}

const app=express();
connectRedis();

app.listen(PORT, () => {
console.log(`Server running on port ${PORT}`);
});
```

Now add Caching logic

```
app.get("/user/:id", async (req, res) => {
const id = req.params.id;

// 1 check cache
const cachedUser = await client.get(`user:${id}`);

if (cachedUser) {
console.log("CACHE HIT");
return res.json(JSON.parse(cachedUser));
}

// 2 fetch from DB
const user = await getUserFromDB(id);

// 3 store in cache
await client.set(`user:${id}`, JSON.stringify(user));
```

```
res.json(user);
});
```

Add TTL (Time To Live)(Very Important)

We usually add expiration.

how long a key should exist before Redis automatically deletes it.

Otherwise cache becomes stale.

Example:

```
await client.set(`user:${id}`, JSON.stringify(user), {
   EX: 60
});
```

Meaning:

Cache expires after 60 seconds

Flow:

DB → Cache (60s) → Expire → DB again

Most Used Redis commands

1. SET

CLI

```
SET user:1 "Prajwal"
```

Node

```
await client.set("user:1", "Prajwal");
```

With TTL

```
await client.set("user:1", "Prajwal", { EX: 60 });
```

Use case

- Caching
- Sessions
- Tokens

2. GET

CLI

```
GET user:1
```

Node

```
const user = await client.get("user:1");
```

Use case

- Fetch cached data

3. DEL

CLI

DEL user:1

Node

await client.del("user:1");

Use case

- Cache invalidation

4. EXISTS

CLI

EXISTS user:1

Node

await client.exists("user:1");

Use case

- Token checks
- Idempotency keys

5. INCR

What it does

Atomically increments number.

CLI

INCR page_views

Node

await client.incr("page_views");

Use case

- Rate limiting
- Counters
- Analytics

INCR only works if the value is **numeric**.

6. HSET

CLI

```
HSET user:1 name "Prajwal" age 22
```

Express

```
await client.hSet("user:1", {
  name: "Prajwal",
  age: 22
});
```

What it does

Stores **object fields** inside Redis.

Use case

- user profiles
- session storage
- settings

7. HGET

CLI

```
HGET user:1 name
```

Express

```
const name = await client.hGet("user:1", "name");
```

What it does

Gets a specific field from hash.

Use case

Fetch one property without loading full object.

8. HGETALL

CLI

```
HGETALL user:1
```

Express

```
const user = await client.hGetAll("user:1");
```

Result:

```
{
  name: "Prajwal",
  age: "22"
}
```

What it does

Fetches full object.

Use case

User session retrieval.

9. LPUSH

CLI

```
LPUSH jobs "send_email"
```

Express

```
await client.lPush("jobs", "send_email");
```

What it does

Push element into **list (queue)**.

Use case

Background job queue.

10. RPOP

CLI

```
RPOP jobs
```

Express Worker

```
const job = await client.rPop("jobs");
```

What it does

Removes job from queue.

Queue flow

Producer → LPUSH
Worker → RPOP

11. RPUSH

CLI

```
RPUSH jobs "send_email"
```

Node

```
await client.rPush("jobs", "send_email");
```

What it does

Adds element **to the right end of list**.

12. LPOP

CLI

```
LPOP jobs
```

Node

```
const job = await client.lPop("jobs");
```

What it does

Removes **first element (left side)** of list.

13. SADD (Set)

CLI

```
SADD online_users user1 user2
```

Node

```
await client.sAdd("online_users", ["user1", "user2"]);
```

Use case

- Unique values
- Online users

What it does

Adds values to **set (unique collection)**.

Example:

```
online_users = {user1, user2, user3}
```

Duplicates not allowed.

Use case

- online users

14.SMEMBERS

CLI

```
SMEMBERS online_users
```

Express

```
const users = await client.sMembers("online_users");
```

What it does

Returns all values in set.

Use case

List active users.

15 ZADD

CLI

```
ZADD leaderboard 100 user1
```

Express

```
await client.zAdd("leaderboard", [
  { score: 100, value: "user1" }
]);
```

What it does

Adds value with **score**.

16.ZRANGE

CLI

```
ZRANGE leaderboard 0 -1 WITHSCORES
```

Express

```
const leaderboard = await client.zRangeWithScores(
  "leaderboard",
  0,
  -1
);
```

What it does

Gets sorted ranking.

Use case

Top players / trending posts.

1️⃣ Cache Invalidation

Problem

Cached data becomes **stale** after DB update.

Example

DB: user.name = "Prajwal"
Cache: "Prajwal"

User updates name → "Rahul"
DB: Rahul
Cache: Prajwal ❌

Best practice

Update DB →Delete cache(Simple + reliable)

Solution 1 — Delete cache on update (most common)

```
// update DB
await db.updateUser(id, data);

// invalidate cache
await client.del(`user:${id}`);
```

Next request:

Cache MISS → DB → new cache

Solution 2 — Update cache

```
await db.updateUser(id, data);

await client.set(`user:${id}`, JSON.stringify(data));
```

Solution 3 — TTL

```
await client.set(key, value, { EX: 60 });
```

Cache auto expires.

2️⃣ Cache Stampede

Problem

When cache expires, **many requests hit DB simultaneously**.

Example

10k requests → same product

Cache expires
↓
10k DB queries ❌

This can **crash DB**.

Solution 1 — Redis Lock

The Idea

Only **ONE request should fetch from DB**.

Others should **wait until cache is filled**.

This is done using a **Redis lock**.

```
const lock = await client.set("lock:user:1", 1, { NX: true, EX: 5 });

if (lock) {
  const data = await db.getUser(1);
  await client.set("user:1", JSON.stringify(data));
}
```

Others wait for cache.

Try to acquire lock

```
const lock = await client.set("lock:user:1", 1, {
 NX: true,
```

```
  EX: 5
});
```

What this means:

NX → Only set if key does NOT exist
EX → Expire after 5 seconds

Who gets the lock?

Only **ONE request succeeds**.

Example:

Request A → lock acquired ✅
Request B → lock failed ❌
Request C → lock failed ❌

Because NX prevents duplicate locks.

Only one request calls DB

For Request A:

```
if (lock) {
  const data = await db.getUser(1);
  await client.set("user:1", JSON.stringify(data));
}
```

Flow:

Request A
↓
Fetch DB (3 seconds)
↓
Store in Redis cache

Now Redis has:

"user:1" → user data

Step 5 — Other requests wait

Requests B and C **did not get the lock**.

So they should **retry cache after a short delay**.

Example idea:

Request B
↓
Wait 100ms
↓
GET user:1
↓
Cache now exists ✅

So they **avoid hitting DB**.

Why EX: 5 is important

EX = expiry time

If the server crashes before releasing the lock:

lock:user:1

Redis will **auto delete after 5 seconds**.

Otherwise the lock could stay forever.

Solution 2 — Stale cache

Serve **old data while refreshing cache**.

User request
↓
Return stale cache
↓
Background refresh

Used by **large systems**.

```
if (cached) {
// return cached data immediately
res.json(JSON.parse(cached));

// refresh cache in background after sending stale cache to user
refreshCache(id, cacheKey);

return;
}
```

There is usually **no perfect way to know if cached data is stale**, so systems use strategies like **TTL, invalidation events, or version checks** as safe mechanisms to refresh data.

3 Cache Penetration

Problem

Requests for **data that does NOT exist**.

Example

GET /user/999999

DB returns null.

Cache never stores it → DB gets hit every time.

Attackers can **flood DB**.

Solution 1 — Cache null

```
if (!user) {
  await client.set(`user:${id}`, "null", { EX: 60 });
}
```

If we cache null for every invalid ID, won't Redis fill up with useless keys?

Yes, that **can happen**, but two things control it.

TTL keeps it temporary

Example:

```
await client.set(`user:${id}`, "null", { EX: 60 });
```

So Redis stores:

user:999999 → null
TTL → 60 seconds

After **60 seconds**, Redis deletes it automatically.

So the memory usage is **temporary**.

Why we still cache null

Without null caching:

Request → user:999999
Cache miss
↓
DB query
↓
User not found

Now imagine a bot hitting:

user:999999
user:999998
user:999997

Every request goes to the **database**.

This is called **Cache Penetration**.

Caching null stops that:

Request → user:999999
↓
Redis returns "null"
↓
No DB call

So we trade **small temporary memory** for **DB protection**.

Real-world improvement

Systems often use **short TTL for nulls**.

Example:

Normal cache TTL → 10 minutes
Null cache TTL → 30 seconds

Solution 2 — Input validation

Reject invalid IDs early

🔥 Cache Breakdown (Hot Key Problem)

Problem

One **very popular key expires**.

Example

product:iphone

Millions of users request it.

When cache expires:

Huge DB traffic spike ❌

Solution 1 — Never expire hot keys

SET product:iphone data

No TTL.

Solution 2 — Background refresh

Cache refreshed before expiry

Solution 3 — Redis lock

Same as **stampede protection**

5️⃣ Write-Through vs Write-Back Cache

These define **how cache and DB stay in sync**.

Write-Through Cache

Flow:

App
↓
Cache
↓
DB

Write operation:

Update Cache
↓
Update DB

Example

```
await client.set(key, value);
await db.update(key, value);
```

Pros

- Cache always consistent

Cons

- Slower writes

Write-Back Cache (Write-Behind)

Flow

App
↓
Cache
↓
Async DB write

Example

```
await client.set(key, value);
queueWriteToDB();
```

DB update happens later.

Pros

- Very fast writes

Cons

- Data loss risk if cache crashes

Write-Around Cache

Flow

Write → DB
Read → Cache

Cache only used for reads.

Most common pattern.

7️⃣ Cache Warming

Preload cache.

Example:

Server starts
↓
Load popular products into Redis

Used for:

Trending products
Homepage data
Configs

Cache Eviction Policies

Redis mostly stores data **in RAM**.

RAM is **limited**.

When Redis memory becomes full:

maxmemory reached

Redis must decide:

Which key should be removed?

This process is called **Cache Eviction**.

Important policies:

1️⃣ LRU — Least Recently Used

Rule:

Remove the key that was **not used for the longest time**.

2️⃣ LFU — Least Frequently Used

Rule:

Remove the key that is **used the least number of times**.

3️⃣ TTL — Expiration

Some keys have an **expiry time**.

Example:

user:1 → expires in 10s
user:2 → expires in 60s
user:3 → expires in 5s

Redis may remove keys that are **about to expire soon**.

This works well when most cache entries already have TTL.

The most useful ones:

Policy	Meaning
allkeys-lru	remove least recently used key
allkeys-lfu	remove least frequently used key
volatile-lru	remove LRU only among keys with TTL
volatile-ttl	remove keys with shortest TTL
noeviction	reject new writes

maxmemory-policy allkeys-lru

How does Redis know which key is LRU or LFU? How does it track it?

Redis stores metadata per key such as last-access time for LRU or access counters for LFU, and uses sampling algorithms to efficiently select eviction candidates when memory is full.

9️⃣ Read vs Write Heavy Systems

Read-heavy systems

Use caching heavily.

Examples:

News websites
Social media
Product catalogs

Write-heavy systems

Caching less effective.

Examples:

Payments
Stock trading

L1,L2,L3 cache

Think of it as **distance from the application**.

L1 → inside app memory
L2 → distributed cache (Redis)
L3 → database

Flow:

Request
↓
L1 Cache (App Memory)
↓
L2 Cache (Redis)
↓
L3 (Database)

L1 Cache (Application Memory)

Where: inside your backend process.

Examples:

- Node.js Map
- Java ConcurrentHashMap
- in-process cache libraries

Example

```
const l1Cache = new Map();

app.get("/user/:id", async (req, res) => {
  const id = req.params.id;

  if (l1Cache.has(id)) {
    return res.json(l1Cache.get(id)); // L1 hit
  }

  const user = await db.getUser(id);

  l1Cache.set(id, user);

  res.json(user);
});
```

Characteristics

Fastest (nanoseconds)
No network
Per-server cache

Problem

Not shared between servers

Example

Server A → cache miss
Server B → cache miss

L2 Cache (Distributed Cache)

Where: external caching system.

Examples:

- Redis
- Memcached

Characteristics

Shared across servers
Very fast (~1ms)
Network call required

L3 (Database)

Final data source.

Examples:

- PostgreSQL
- MySQL
- MongoDB

Characteristics

Slowest
Persistent storage
Source of truth

Why not redis as db ..?

Redis can technically be used as a primary database because it supports persistence mechanisms like RDB snapshots and AOF logs. However, in most real-world systems it is mainly used as a caching layer rather than the main database. The primary reason is that Redis stores data in RAM, which is significantly more expensive than disk storage used by traditional databases. This makes it impractical to store large amounts of permanent data in Redis. Additionally, while Redis offers persistence, it does not provide the same level of durability and strong guarantees as traditional databases. Systems like relational databases also support complex queries, indexing, joins, and relational data modeling, which Redis is not designed for since it follows a simple key–value structure. Because of these limitations, Redis is typically placed in front of a persistent database to cache frequently accessed data, reduce database load, and provide extremely low-latency responses. In this architecture, the database remains the source of truth while Redis acts as a high-speed cache to improve performance and scalability.

Redis as QUEUE

A **queue** is a data structure that follows **FIFO**.

FIFO = First In First Out

2️⃣ Why We Use Queues in Backend

Some tasks should **not block the API response**.

Example:

User registers
↓
Send welcome email

Bad approach:

API waits until email is sent ❌

Good approach:

API pushes job to queue
↓
Worker processes job later

Flow:

Client
↓
API
↓
Redis Queue
↓
Worker
↓
Process job

Typical tasks:

Email sending
Image processing
Notifications
Video processing
Payment retries
Reports generation

3️⃣ Why Redis Works Well for Queues

Redis is:

very fast
in-memory
atomic operations

Operations like:

LPUSH
RPUSH
LPOP
RPOP

make queues easy.

4️⃣ Redis Queue Pattern

Most common pattern:

Producer → RPUSH
Worker → LPOP

1️⃣ Using LPOP on an empty list

LPOP myqueue

- If myqueue **has elements**, LPOP removes and returns the **leftmost element**.

- If myqueue is **empty** or does **not exist**, LPOP returns:

nil

✅ That's it — **no error is thrown**, just nil (or null in Node.js/JS client).

Problem

- Worker needs tasks from a Redis queue.
- Using LPOP on an empty queue returns null → worker must **poll repeatedly**.
- Polling wastes **CPU and network resources**, especially if tasks arrive sporadically.

Solution:

- Use BLPOP (blocking pop).
- Worker **waits efficiently** until a task is available → no polling needed.

Why:

- Immediate processing of new tasks.
- Saves system resources.
- Ideal for background jobs and real-time queues.

BLPOP — Blocking pop

BLPOP = "Blocking LPOP".

If the list is empty, Redis **waits until an element arrives** (or until a timeout).

```
client.blPop("tasks", 0);
```

key → the list(s) you want to pop from

timeout → **how long Redis should wait if the list is empty**

What 0 means

- 0 = **wait forever** until an element appears in the list
- Any number n > 0 = wait up to n seconds, then return nil if the list is still empty

◇ Redis Blocking & Single Client Issue (Notes)

Problem:

- Using **a single Redis client** for both **normal commands** (rPush, get, set) and **blocking commands** (BLPOP, BRPOP).
- **BLPOP is blocking** — it waits until a job exists.
- Single client cannot execute another command while blocked → **API requests hang/stuck**.

Symptoms:

- /upload or rPush appears **stuck** even though Redis is connected.
- Worker running in same process causes **request never returns**.

Solution:

1. **Use separate Redis clients**:

Client	Purpose
API Client	rPush, get, set (non-blocking)
Worker Client	BLPOP (blocking)

2. **Flow with two clients**:

API Client: rPush job → Redis → Worker Client: BLPOP receives job → processes job

- API client never blocked by worker.
- Worker client can safely block on queue.

Takeaway / Rule of Thumb:

- **Never mix blocking and non-blocking commands on the same Redis client**.
- Always have **separate client connections** for blocking operations.

Redis as a Pub/Sub?

- **Pub/Sub** = **Publisher / Subscriber pattern**
- Redis allows clients to **publish messages to a channel**
- Other clients **subscribe to the channel** to receive messages in real-time
- **Fire-and-forget** — messages are **not stored** in Redis (unlike a queue)

Use case examples:

- Real-time notifications (chat messages, alerts)
- Event broadcasting (system events to multiple services)
- Live updates to front-end (scores, stock prices, dashboards)

2️⃣ Architecture

Publisher → Redis → Channel → Subscribers

- **Publisher**: sends messages
- **Redis Channel**: acts as a topic / stream
- **Subscriber(s)**: listen to messages and react

3️⃣ Key Commands / Methods

CLI Command	Redis Client (Node.js)	Description
PUBLISH channel message	client.publish("channel", message)	Sends message to channel
SUBSCRIBE channel	subscriber.subscribe("channel", callback)	Listen for messages

CLI Command	Redis Client (Node.js)	Description
UNSUBSCRIBE channel	subscriber.unsubscribe("channel")	Stop listening

Important: Publisher and Subscriber **must use separate Redis clients** — Pub/Sub connections **cannot share commands** like queues do.

Single Subscriber Flow

1. Start subscriber → connects Redis → subscribes to channel
2. Publisher publishes → subscriber receives message → processes/logs it

Publisher → Redis channel → Subscriber

- Works for **one subscriber** at a time.

redisPublisher.ts — Publisher Client

```
import { createClient } from "redis";

export const publisherClient = createClient();

publisherClient.on("error", (err) => console.log("Publisher Redis Error:", err));

await publisherClient.connect(); // Connect to Redis
console.log("Publisher connected");
```

✅ This client will **only publish messages**.

redisSubscriber.ts — Subscriber Client

```
import { createClient } from "redis";

export const subscriberClient = createClient();

subscriberClient.on("error", (err) => console.log("Subscriber Redis Error:", err));
```

```
await subscriberClient.connect(); // Connect to Redis
console.log("Subscriber connected");
```

This client will **only subscribe** to Redis channels.

2️⃣ Subscriber Worker

subscriber.ts

```
import { subscriberClient } from "./redisSubscriber";

// Subscribe to a channel
async function startSubscriber() {
  console.log("Subscriber started, listening to 'posts_notifications'");

  await subscriberClient.subscribe("posts_notifications", (message) => {
    try {
      const post = JSON.parse(message); // Parse the JSON message
      console.log("Received post:", post.title, "| ID:", post.postId);
    } catch (err) {
      console.error("Failed to parse message:", message, err);
    }
  });
}

startSubscriber();
```

Explanation:

1. Connects to Redis → ready to listen
2. Subscribes to "posts_notifications"
3. Every published message triggers callback → logs the post

For multiple subscribers, just run **this script in multiple terminals or processes**. Each will get every message.

3️⃣ Publisher API

server.ts

```
import express from "express";
import { publisherClient } from "./redisPublisher";
import { v4 as uuidv4 } from "uuid";
const app = express();
app.use(express.json());

app.post("/create-post", async (req, res) => {
  const { title } = req.body;
  if (!title) return res.status(400).json({ error: "Title is required" });

  const post = {
    postId: uuidv4(),
    title,
    createdAt: Date.now(),
  };
  try {
    // Publish message to Redis channel
    await publisherClient.publish("posts_notifications", JSON.stringify(post));
    res.json({ message: "Post created and notification sent", post });
  } catch (err) {
    console.error("Publish error:", err);
    res.status(500).json({ error: "Failed to send notification" });
  }
});

app.listen(5000, () => console.log("Publisher API running on port 5000"));
```

Explanation:

1. Receives /create-post requests
2. Generates a **unique ID** for the post
3. Publishes the post to the Redis channel
4. Returns **immediate response** → non-blocking

Redis as Rate Limiter

Rate limiting controls how often a user/client can make a request to your server.

Purpose: prevent abuse, DoS attacks, brute-force attempts.

Example: “Max 5 requests per minute per IP”

🔹 2️⃣ Why Redis for Rate Limiting?

- Redis is **in-memory** → super fast.
- Supports **atomic increments** → avoids race conditions.
- Can **set expiry on keys** → naturally supports time windows.
- Works across **distributed systems** → all servers share the same Redis.

🔹 3️⃣ Rate Limiting Strategies

1. **Fixed Window Counter**
    - Count requests per fixed time window (e.g., 10 requests per minute)
    - Easy, but bursts at window edges possible
2. **Sliding Window Log**
    - Track timestamps of each request → more precise
    - Uses more memory
3. **Token Bucket / Leaky Bucket**
    - Tokens represent requests → refill over time
    - Smooth rate limiting → prevents bursts

🔷 Redis Commands Used

Command	Usage in Rate Limiting
INCR key	Increment request count
EXPIRE key seconds	Set TTL for window
GET key	Optional: read current count
DEL key	Optional: reset count

```
import express, { Request, Response, NextFunction } from "express";
import { createClient } from "redis";

const app = express();
app.use(express.json());

const redisClient = createClient();
await redisClient.connect();

const WINDOW_SIZE = 60; // seconds
const MAX_REQUESTS = 5; // max requests per window

// Rate Limiter Middleware
async function rateLimiter(req: Request, res: Response, next: NextFunction) {
  const ip = req.ip; // or use user ID
  const key = `rate:${ip}`;
```

```
  const current = await redisClient.incr(key); // increment count atomically

  if (current === 1) {
    // first request → set expiry
    await redisClient.expire(key, WINDOW_SIZE);
  }

  if (current > MAX_REQUESTS) {
    return res.status(429).json({ message: "Too many requests. Try later." });
  }

  next();
}

app.use(rateLimiter);

app.get("/", (req, res) => {
  res.send("Hello! You are within the limit.");
});

app.listen(5000, () => console.log("Server running on port 5000"));
```

- 6️⃣ **How This Works**

1. Client makes request → middleware runs

2. INCR increases the request count in Redis **atomically**
3. First request → set expiry (EXPIRE) for 1-minute window
4. If count exceeds max → return 429 Too Many Requests
5. Otherwise → allow the request

Works **across distributed servers** if all share the same Redis client

---

🔷 7️⃣ Advantages of Redis Rate Limiting

- **Fast** → in-memory counters
- **Atomic** → avoids race conditions
- **Flexible** → per-IP, per-user, per-API key
- **Distributed ready** → multiple servers can enforce limits consistently